基于 Clang/LLVM 的语言处理流程与 SysY 手写实现

作者：LIZZYGREAT

2026 年 10 月 7 日

摘要

本文以一个含函数、while、if/else、取模和整数输入输出的 SysY 风格程序为贯穿案例，使用 Clang 14 观察预处理、词法/语法/语义分析与 IR 生成、x86-64 汇编、ELF 目标文件和显式链接的完整过程，并通过实际 `readelf` 输出核对节区、符号与重定位项。随后根据源程序语义独立手写 LLVM IR 与 RV64 汇编，对照控制流图与栈帧布局逐条说明设计意图，并分别链接运行库验证。六组输入在宿主机 reference、手写 LLVM 和 QEMU/RISC-V 上均得到 1、5、8、16、21、85。优化对比显示，Clang -O2 消除了该模块中的 alloca/load/store，并在 x86-64 上生成四 lane 向量运算；RV64 目标保留标量循环并内联 calc。最后依据 LLVM 与 AscendNPU-IR 官方材料分析 VecAdd 的多层 MLIR lowering，并解读官方 PlanMemory 测试用例；该部分为资料与源码分析，没有运行本机 Ascend 编译器或设备。全部实验由脚本驱动，可逐阶段复现。

关键词：编译器；Clang；LLVM IR；RISC-V；ABI；MLIR；lowering

1 引言

编译器把源程序逐步转换为可执行程序。理解每一步的输入、输出和符号关系，有助于分清预处理、编译、汇编和链接的职责。LLVM IR 适合观察语言结构如何变为控制流和算术指令；RISC-V 汇编则展示调用约定、寄存器和栈帧如何落实到目标架构。

本实验固定一个程序，避免不同算法导致比较基准变化。程序令 i 从 1 递增到输入 n，若 i 为偶数则将 $2i$ 加入 sum，否则将 i 加入 sum，最后输出 sum。程序覆盖局部整型变量、赋值、加法、乘法、取模、比较、条件分支、循环、函数参数、函数返回和运行库调用。所有实现都使用同一输入集合验证。

全文围绕四个问题组织。第一，预处理、编译、汇编、链接各自做了什么，产物之间是什么关系（第 3 节）；第二，源程序的条件与循环如何变成 LLVM 基本块和 `br`，又如何能独立手写出一份等价 IR（第 4 节）；第三，同一逻辑在 RV64 上如何落到寄存器、栈帧和调用约定（第 5 节）；第四，优化级别和抽象层次的变化会带来什么（第 7、8 节）。所有结论都来自本机真实产物，原始命令、输出和中间文件分别保存在 `scripts/`、`logs/` 和 `records/` 中，第 9 节给出复现入口。

2 实验对象与工具环境

2.1 贯穿程序

整次作业只使用一个程序，避免更换算法导致比较基准漂移。其 SysY 源程序 `source/example.sy` 如下。

```
int calc(int n) {
    int i = 1;
    int sum = 0;
    while (i <= n) {
        if (i % 2 == 0) { sum = sum + i * 2; }
        else            { sum = sum + i;     }
        i = i + 1;
    }
    return sum;
}

int main() {
    int n;      n = getint();
    int result; result = calc(n);
    putint(result);
    return 0;
}
```

该程序覆盖局部整型变量、赋值、整型常量、加法、乘法、取模、比较、`if/else`、`while`、函数定义与调用、参数传递、返回值以及运行库输入输出，足以观察编译器的主要结构。语义为：i 从 1 递增到 n，i 为偶数时累加 $2i$，否则累加 i，最后输出 sum。为使 Clang 能直接处理，`source/example_clang.c` 与 `example.sy` 算法完全一致，仅补充运行库声明 `int getint(void);` 与 `void putint(int x);`；宿主机验证所需的 `getint/putint` 由 `source/host_runtime.c` 以 `scanf/printf` 实现。固定测试集为 $n = 1, 2, 3, 4, 5, 10$，期望输出依次为 $1, 5, 8, 16, 21, 85$；该期望值独立于任何实现，作为唯一的 Ground Truth。

2.2 工具环境

工作环境为 Ubuntu 22.04.5 on WSL2（内核 `6.18.33.2-microsoft-standard-WSL2`），全部工具版本记录于 `logs/00_environment.log`。

表 1: 本实验使用的工具链

工具	版本	本实验用途
Clang / LLVM	14.0.0	预处理、LLVM IR、x86-64 汇编、目标文件；接受手写 `.ll`
GCC	11.4.0	提供系统链接器驱动环境
RISC-V unknown-elf GCC	15.1.0	汇编、链接手写 RV64 汇编与课程 runtime
QEMU	7.0.0	用户态运行 RISC-V 可执行文件
Flex / Bison	2.6.4 / 3.8.2	课程编译器环境既有组件
Tectonic	0.16.9	编译本报告 LaTeX 源文件

课程目录 `SysY_Compiler_2025` 已提供编译器与 SysY runtime，本实验只复用其 `lib/libsysy_riscv.a`，没有重新构建或改动该目录。Clang 的 RISC-V 参考目标与手写目标统一使用 `rv64gc/lp64d`，与 runtime 的 ELF 架构标志一致。

3 Clang 语言处理流程

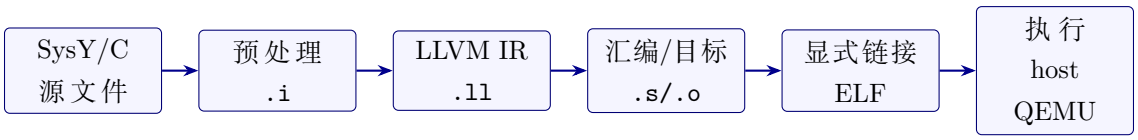


图 1: 从源程序到运行文件的实验流程

3.1 预处理

命令 `clang -E source/example_clang.c -o generated/example.i` 生成预处理文件。预处理器只处理以 `##` 开头的指令：展开宏、插入 `##include` 内容、求值条件编译、处理行标记。它按行替换文本，不做词法之外的语义判断，因此不理会 `while`、`if` 的含义。本例没有宏、条件编译和头文件，`generated/example.i` 只有 11 行 `##` 行标记，其后语句与输入完全相同。

```
# 1 "source/example_clang.c"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 361 "<built-in>" 3
...
# 1 "source/example_clang.c" 2
int getint(void);
void putint(int x);

int calc(int n) {
    int i = 1;
    ...
```

行标记形如 `## 行号" 文件名" 标志`，供后续阶段和调试器把生成代码映射回源文件；`<built-in>` 是编译器内建的定义集合。除这 11 行标记外，`.i` 内容与输入逐字相同——这恰好从反面说明预处理阶段“没有做什么”：它不建立符号表、不检查类型、也不理解控制流，这些都属于下一节的编译阶段。

3.2 编译器内部各阶段

`clang -cc1` 是真正的编译器前端。对同一份源文件，它依次完成以下工作；本实验通过在 `example.ll` 中看到的结果反推各阶段产物。

表 2: 编译器内部阶段及其在本程序中的表现

阶段	职责	本程序的观察
词法分析	把字符流切成记号	`int`、`calc`、`i`、`<=`、`100` 等记号
语法分析	按文法构造抽象语法树	得到 `while`、`if/else` 的嵌套结构
语义分析	类型检查、建立符号表、插入隐式转换	判定 `i <= n` 为 `i32` 比较，`calc(n)` 类型匹配
IR 生成	遍历 AST 生成 LLVM IR	局部变量变为 `alloca`，语句变为基本块与 `br`
中端优化	在 IR 上做与目标无关的变换	`-O0` 不做；`-O2` 提升为 SSA 与 `phi`（第 7 节）
后端	指令选择、寄存器分配、指令调度	生成 x86-64 或 RV64 汇编

其中前三个阶段常合称“前端”，它们只依赖源语言；中端依赖 IR；后端依赖目标架构。这与图 1 的整体流程互相印证：预处理器独立于前端，前端到 IR 为止，中端和后端才与目标相关。

3.3 LLVM IR 生成

命令如下：

```
clang -S -emit-llvm -O0 source/example_clang.c -o generated/example.ll
```

生成 `generated/example.ll`。它是与目标无关的中间表示，函数、类型、指令都在此层显式可见。`-O0` 下一个 `alloca` 对应一个局部变量，每次读写都显式 `load/store`，见第 4 节。

3.4 汇编与目标文件

`clang -S -O0` 生成宿主机汇编 `generated/example_x86.s`；`clang -c -O0` 继续调用集成汇编器，将汇编编码为机器码，产出 ELF64 可重定位目标文件 `generated/example.o`。汇编器负责指令编码、地址分配和符号表构造，此时地址尚未最终确定，因此外部引用以重定位项的形式保留。对 `example.o` 的实际检查结果如下。

表 3: example.o 的 ELF 特征（file 与 readelf 实际输出）

检查项	实际结果
文件类型	ELF 64-bit LSB relocatable, x86-64, not stripped，即 ET_REL
主要节区	.text（162 字节）、.rela.text、.eh_frame、.symtab、.strtab
已定义符号	calc（FUNC，100 字节）、main（FUNC，50 字节）
未定义符号	getint、putint，均为 UND
重定位项	.rela.text 中 3 条 R_X86_64_PLT32，目标为 getint、calc、putint

readelf -h 给出的 ELF 文件头也印证了这一点：

```
Class:                             ELF64
Type:                              REL (Relocatable file)
Machine:                           Advanced Micro Devices X86-64
Entry point address:               0x0
Start of program headers:          0 (bytes into file)
```

Type 为 REL 而非 EXEC/DYN，Entry point 为 0x0，且程序头数量为 0，说明该文件只是“待合并的部件”：地址未定、没有进程映像布局，无法被内核直接加载执行。两处细节值得注意。其一，符号表把 calc/main 标为 FUNC 且带字节大小，而 getint/putint 标为 UND，直观展示了“本文件提供什么”与“本文件还欠什么”。其二，重定位项中除外部符号外还出现了 calc——本机 Clang 默认使用 PIE，模块内调用同样经 PLT 重定位，其最终地址由链接器确定，这也解释了为什么 .rela.text 中有 3 条记录而不是 2 条。

3.5 链接

本实验分别生成 example.o 与 host_runtime.o，再用 Clang 驱动显式链接。日志显示 Clang 调用 /usr/bin/ld，并加入 CRT 启动对象和 -lc。链接器按符号名把各目标文件的定义与引用配对：host_runtime.o 定义 getint/putint，解决 example.o 的两个未定义符号；系统 C 库解析 runtime 中的 scanf/printf。随后合并同名节区、按最终地址应用重定位、丢弃不再需要的符号信息，生成可执行 ELF generated/reference_linked，其六组输出与 Ground Truth 一致。反过来说，单独的 example.o 既缺少两个外部函数定义，也没有进程启动代码，因此不能独立完成整个程序功能；getint/putint 正是在链接阶段被解析的。

表 4: 语言处理阶段及本实验观察

阶段	命令或产物	本实验观察
预处理	`clang -E`; `example.i`	保留 C 语句并带行标记；本例无宏/头文件展开
LLVM IR	`clang -S -emit-llvm -O0`; `example.ll`	alloca、load/store、icmp、br、call、ret
汇编	`clang -S -O0`; `example_x86.s`	x86-64 指令和调用序列
汇编器	`clang -c`; `example.o`	ELF REL、节区、符号和重定位
链接器	`example.o + host_runtime.o`	解析 runtime/C 库符号并生成可执行 ELF

4 LLVM IR 控制流与手写实现

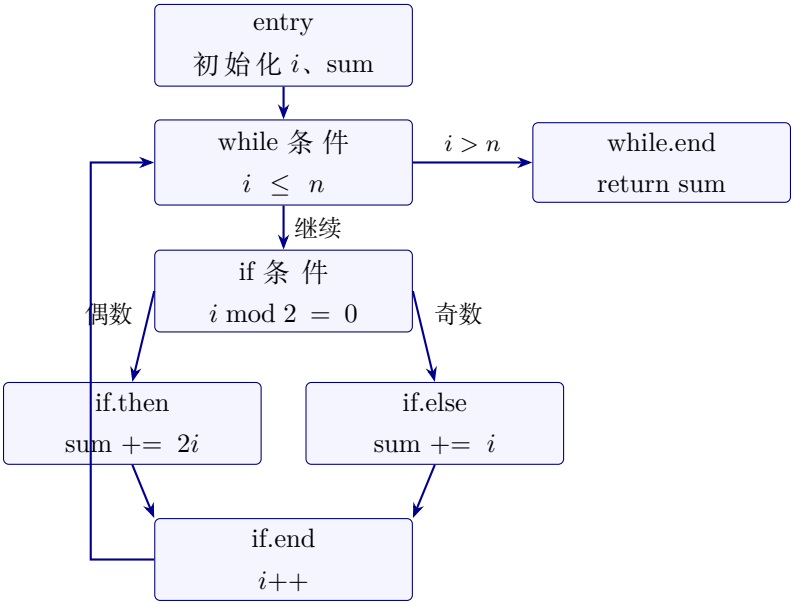


图 2: `calc` 的基本块与循环回边

Clang -O0 生成的 calc 使用数字标签。入口块初始化 i 和 sum 后进入 `%5`；`%5` 用 `icmp sle` 计算 $i \le n$。真分支进入 `%9` 判断 $i \bmod 2$，两个分支 `%13`、`%18` 分别计算偶数项和奇数项，并汇入 `%22`。`%22` 执行 i++ 后无条件跳回 `%5`，构成 while 回边；假分支到 `%25` 读取 sum 并返回。

表 5: 源程序结构与 LLVM 指令对应

源程序结构	LLVM 指令	作用
局部变量声明/赋值	alloca、store	为 n/i/sum 分配栈槽并写初值或新值
读取变量	load	取 i、n、sum 的当前值
加法、乘法、取模	add、mul、srem	计算计数器、累加项与奇偶性
比较	icmp sle、icmp eq	判断循环范围和余数是否为零
if/while	br、基本块	根据条件转移，合并分支并形成循环回边
函数关系	define、call、ret	定义 calc/main，调用 getint/calc/putint 并返回

4.1 手写 LLVM IR

手写 `llvm/manual.ll` 不复用 Clang 的输出，而是按上面的 CFG 直接组织命名基本块。与自动生成版本的差异恰好说明了“人工设计”与“工具默认策略”的区别：块名是语义化的 `while.cond` 而不是 `%5`；没有 `attributes`、`!llvm.loop` 等约定性元数据；`main` 中直接用 SSA 值传递 `getint` 与 `calc` 的返回值，不再绕经栈槽。控制流与算术指令则与源程序逐项对应。

```
declare i32 @getint()
declare void @putint(i32)

define i32 @calc(i32 %n) {
entry:
  %i.addr = alloca i32, align 4
  %sum.addr = alloca i32, align 4
  store i32 1, i32* %i.addr, align 4      ; i = 1
  store i32 0, i32* %sum.addr, align 4    ; sum = 0
  br label %while.cond

while.cond:
  %i.cond = load i32, i32* %i.addr, align 4
  %continue = icmp sle i32 %i.cond, %n    ; i <= n
  br i1 %continue, label %if.cond, label %while.end

if.cond:
  %i.parity = load i32, i32* %i.addr, align 4
  %remainder = srem i32 %i.parity, 2      ; i % 2
  %is.even = icmp eq i32 %remainder, 0
  br i1 %is.even, label %if.then, label %if.else

if.then:
  %sum.even = load i32, i32* %sum.addr, align 4
  %i.even = load i32, i32* %i.addr, align 4
  %twice.i = mul nsw i32 %i.even, 2       ; 2 * i
  %sum.even.next = add nsw i32 %sum.even, %twice.i
```

```
  store i32 %sum.even.next, i32* %sum.addr, align 4
  br label %if.end

if.else:
  %sum.odd = load i32, i32* %sum.addr, align 4
  %i.odd = load i32, i32* %i.addr, align 4
  %sum.odd.next = add nsw i32 %sum.odd, %i.odd
  store i32 %sum.odd.next, i32* %sum.addr, align 4
  br label %if.end

if.end:
  %i.current = load i32, i32* %i.addr, align 4
  %i.next = add nsw i32 %i.current, 1     ; i = i + 1
  store i32 %i.next, i32* %i.addr, align 4
  br label %while.cond                    ; while 的回边

while.end:
  %sum.result = load i32, i32* %sum.addr, align 4
  ret i32 %sum.result
}

define i32 @main() {
entry:
  %n = call i32 @getint()
  %result = call i32 @calc(i32 %n)
  call void @putint(i32 %result)
  ret i32 0
}
```

while 没有也不需要有专门的 LLVM 指令：while.cond 计算条件、br i1 二选一，if.end 的无条件 br 跳回 while.cond 即构成回边。if/else 同样只是 if.cond 处的一次条件 br 加两个分支块，两条分支在 if.end 汇合。Clang 接受该 IR，将其与 host_runtime.c 链接为 generated/manual_llvm，六组输入全部通过（outputs/manual-llvm-test.txt）。

表 6: 自动生成 IR 与手写 IR 的组织方式对比

对比项	generated/example.ll（Clang -O0）	llvm/manual.ll（手写）
基本块命名	数字标签 %5、%9、%13	语义名 while.cond、if.then、if.end
局部变量	三个 alloca，含形参 n 的栈槽	只给 i、sum 两个 alloca，n 直接用 SSA 形参
分支合流	%13、%18 汇入 %22	if.then、if.else 汇入 if.end
元数据	带 attributes、!llvm.loop、!llvm.module.flags	不产生任何元数据
目标三元组	x86_64-pc-linux-gnu 与完整 datalayout	仅声明 target triple
指令集合	同一组核心指令	逐条对应源语句，无新增或缺失的操作

两份 IR 使用的指令集合完全相同——alloca、load、store、add、mul、srem、icmp、br、call、ret 逐条对应，既没有新增也没有缺失的操作。表 6 中“指令集合”一行是关键：两份 IR 的语义操作完全一致，差别只在命名、元数据和值表示的组织方式。这说明 -O0 下 LLVM IR 到源程序的映射是相对直接的，也说明手写实现可以在保留全部语义的前提下采用更适合人阅读的组织形式。

5 RISC-V 汇编与 ABI

5.1 参考汇编与调用约定

RISC-V 参考汇编由 Clang 14 交叉生成（--target=riscv64-unknown-elf -march=rv64gc -mabi=lp64d），先用于观察成熟后端如何表达同一逻辑，再作为手写实现的对照。其 calc 序言为：

```
calc:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)
    addi s0, sp, 32
    sw   a0, -20(s0)
    ...
.LBB0_1:
    lw   a1, -24(s0)      # i
    lw   a0, -20(s0)      # n
    blt  a0, a1, .LBB0_6
    ...
.LBB0_2:
    lwu  a0, -24(s0)      # i % 2 的替代实现
    srli a1, a0, 31
```

```
    add  a1, a1, a0
    andi a1, a1, -2
    subw a0, a0, a1
    bne  a0, zero, .LBB0_4
```

把这段与下一节的手写版本对照，可以得到三个具体结论。第一，栈帧完全一致：Clang 同样用 32 字节帧、ra 存 24(sp)、s0 存 16(sp)、三个局部量放在 -20/-24/-28(s0)，说明手写实现在没有照抄的情况下收敛到了与真实后端相同的布局。第二，取模的实现不同：RV64 没有硬件取余指令，remw 需要微码或库调用，因此 Clang 把 $i \bmod 2$ 展开成 lwu/srli/add/andi/subw 五条定点运算；手写版本直接用 remw 加 beqz，更贴近源程序语义但指令条数更少，这也说明“指令少”不等于“更快”。第三，*main* 的帧大小不同：Clang 的 main 用 48 字节，因为它把返回值先 sd 溢出到栈再取出；手写 main 只用 16 字节，返回值始终留在 a0 中直接传递。

表 7: 本实验用到的 RISC-V 寄存器与调用约定

寄存器/规则	本实验用法
a0	第一个整型参数与整型返回值；getint、calc、putint 之间传值
t0-t2	calc 内的 caller-saved 临时值；本实验只用这三个
s0	calc 的 frame pointer；属被调用者保存，成对保存/恢复
sp	栈指针；calc/main 分别保留 32/16 字节且保持 16 字节对齐
ra	返回地址；calc 与 main 跨越函数调用保存/恢复

5.2 栈帧布局

RV64 调用约定要求 sp 16 字节对齐。calc 的栈帧共 32 字节：顶部保存返回地址与旧的 s0，中部放三个局部变量。堆栈从高地址向低地址增长，因此“入栈”即减小 sp。

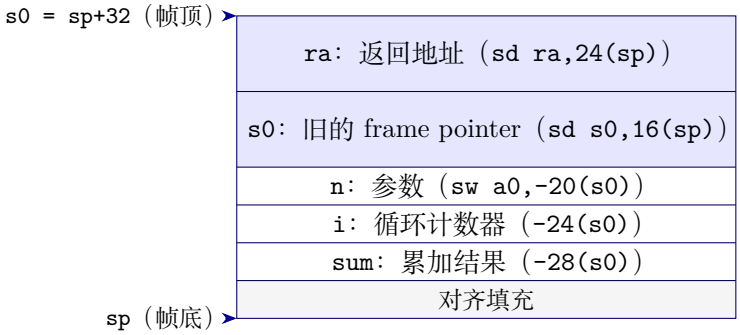


图 3: calc 的 32 字节栈帧（addi sp,sp,-32）

5.3 手写汇编

手写 riscv/manual.s 独立遵守同一约定：序言建立栈帧并保存 ra/s0，循环条件用 blt 在 i > n 时退出，奇偶判断用 remw 加 beqz，偶数项用 slliw 左移一位代替乘 2，累加用 addw，循环尾递增 i 后无条件跳回条件块。控制流标签与第 4 节的 LLVM 基本块一一对应。

```
calc:
    addi sp, sp, -32
    sd   ra, 24(sp)
    sd   s0, 16(sp)
    addi s0, sp, 32
    sw   a0, -20(s0)        # n = 入参
    li   t0, 1
    sw   t0, -24(s0)        # i = 1
    sw   zero, -28(s0)      # sum = 0
.Lwhile_cond:
    lw   t0, -24(s0)        # i
    lw   t1, -20(s0)        # n
    blt  t1, t0, .Lwhile_end    # if (n < i) 退出循环
.Lif_cond:
    li   t1, 2
    remw t2, t0, t1         # i % 2
    beqz t2, .Lif_then
    j    .Lif_else
.Lif_then:
    lw   t0, -28(s0)        # sum
    lw   t1, -24(s0)        # i
    slliw t1, t1, 1         # 2 * i
    addw t0, t0, t1
    sw   t0, -28(s0)
    j    .Lif_end
.Lif_else:
    lw   t0, -28(s0)
    lw   t1, -24(s0)
    addw t0, t0, t1
    sw   t0, -28(s0)
.Lif_end:
    lw   t0, -24(s0)
    addiw t0, t0, 1         # i + 1
    sw   t0, -24(s0)
    j    .Lwhile_cond       # while 的回边
.Lwhile_end:
    lw   a0, -28(s0)        # 返回值放入 a0
    ld   s0, 16(sp)
    ld   ra, 24(sp)
    addi sp, sp, 32
    ret
```

main 只保留 16 字节栈帧用于存放 ra，连续调用 getint、calc、putint，三者之间的整数传递全部通过 a0 完成；这体现了 “参数与返回值共用 a0 “这一约定。

5.4 链接与运行

手写汇编由 `riscv64-unknown-elf-gcc` 汇编，并静态链接课程已有的 SysY runtime 归档 `libsysy_riscv.a`，沿用课程 `toolchain.conf` 与 `asm2bin.sh` 的配置：`-static -mcmodel=medany -Wl,--no-relax,-Ttext=0x90000000`。产物 `generated/manual_riscv` 是 RISC-V ELF 可执行文件，入口地址 `0x90000044`，由 `qemu-riscv64` 用户态模拟执行。runtime 的析构函数会向 stderr 打印 `TOTAL: 0H-0M-0S-0us` 计时摘要，测试脚本只比较标准输出，因此该摘要不影响数值判定。

6 运行验证

表 8: 六组输入在三种实现上的输出比较

输入 n	Reference	手写 LLVM	手写 RISC-V/QEMU	结果
1	1	1	1	PASS
2	5	5	5	PASS
3	8	8	8	PASS
4	16	16	16	PASS
5	21	21	21	PASS
10	85	85	85	PASS

表 8 的数字来自脚本 `scripts/09\_final\_validation.sh` 的实际输出。测试框架是 `source/test\_reference.sh`：它把六组输入期望值硬编码为一个列表，逐组把输入喂给被测可执行文件，再与期望值比较——注意不是与另外两个实现的输出比较。这一点很重要：三路实现互相一致但集体出错的情况不会被漏掉，只有三者都等于 Stage 1 固定的 Ground Truth 才算通过。

各阶段结果分别留证于 `outputs/` 下的 `reference-test.txt`、`manual-llvm-test.txt` 与 `manual-riscv-test.txt`，汇总表为 `final-validation.md`，完整命令输出见 `logs/09_final_validation.log`。其中 RISC-V 一列由 `qemu-riscv64` 执行，课程 runtime 会在每次运行结束后向标准错误输出 `TOTAL: 0H-0M-0S-0us` 计时摘要；比较脚本只取标准输出，因此该摘要保留在日志中而不参与判定。

三份实现的栈帧、寄存器使用和指令选择各不相同，却得到完全相同的数值结果，这说明前面的 CFG 分析、ABI 说明和手写实现确实保持了源程序的语义等价性。

7 优化观察：-O0 与 -O2

对照产物由同一脚本生成：`exploration/O0.ll`、`O2.ll`（LLVM IR），`O0.s`、`O2.s`（x86-64），`O0\_riscv.s`、`O2\_riscv.s`（RV64），三份 unified diff 保存在 `outputs/optimization-*.diff`。以下只描述当前程序真实发生的变化。

表 9: LLVM IR 指令数量与结构变化

指标	-O0	-O2
`alloca`	6	0
`load`	11	0
`store`	9	0
`phi`	0	32
局部变量表示	栈槽 + 内存读写	SSA 临时值 + 循环 `phi`
奇偶判断	`srem` + `icmp eq` + `br`	`and`/`xor`/`shl` 位运算
`main` 对 `calc` 的调用	存在 `call @calc`	已内联；`@calc` 定义因外部可见而保留

7.1 从内存槽到 SSA

`-O0` 把每个局部变量放进取址栈槽，任何读写都要经过 `load`/`store`。`-O2` 在 IR 上执行 mem2reg 一类变换后，可变变量成为 SSA 值；循环中跨迭代传递的量则用 `phi` 节点表达。本例最直观的证据是 `alloca`/`load`/`store` 全部归零、`phi` 从 0 增至 32。这解释了为什么“优化”首先改变的不是指令条数，而是值的表示方式：寄存器化之后，后续的算术化简和向量化才有作用对象。

7.2 奇偶判断被改写成位运算

源程序用 `i % 2 == 0` 选择分支。`-O2` 识别出这是奇偶测试，不再依赖取模。x86-64 的 IR 中可以看到 `and`/`xor`/`shl` 的组合，RV64 的 `calc` 则压缩为：

```
calc:
    li   a1, 0
    blez a0, .LBB0_3
    li   a2, 1
.LBB0_2:
    mv   a3, a2
    not  a2, a2
    andi a2, a2, 1
    sllw a2, a3, a2        # i << (i 为偶数 ? 1 : 0)
    addw a1, a1, a2        # sum += 该项
    addiw a2, a3, 1
    blt  a3, a0, .LBB0_2
.LBB0_3:
    mv   a0, a1
    ret
```

这段代码值得逐行读。`not`/`andi` 取出 i 的奇偶位再取反，得到“i 为偶数时为 1，否则为 0”的移位量；`sllw a2, a3, a2` 做变长左移，一次性完成原程序的两个分支——偶数时左移 1 位即乘 2，奇数时左移 0 位即原值。整个 `if/else` 因此消失，只剩一个循环。此外 `calc` 在 `-O2` 下

没有建立任何栈帧（无 sp 调整、无 ra 保存），因为优化后它已不含函数调用，成为叶子函数。

7.3 两种目标的低层差异

尽管 IR 是目标无关的，同一份 -O2 IR 在两种后端上继续下降的结果并不相同。

x86-64 带 SSE2，且本程序的循环次数可判定，因此 loop vectorizer 把它改写为四 lane 运算：O2.ll 中出现 <4 x i32> 类型的 phi 与 shl，以及 call i32 @llvm.vector.reduce.add.v4i32，汇编中可见 XMM 寄存器和 paddd。由于 calc 不是 4 的整数倍长度也能正确退出，向量化版本还额外生成了边界与尾部路径。

RV64 目标为 rv64gc，不含 RVV 向量扩展，因此 O2_riscv.s 仍是标量循环，没有向量寄存器指令；优化所做的只是内联与算术化简。两种目标的共同点是 main 都不再调用 calc，即函数内联在两条路径上都发生了。

表 10: -O0 与 -O2 汇编规模变化

汇编文本行数	-O0	-O2	变化来源
x86-64（.s）	87	385	向量化循环体展开与 calc 内联
RV64（.s）	91	56	消除栈帧读写、分支退化为变长移位

两行行数朝相反方向变化，说明汇编行数不能作为优化效果的度量：x86-64 因为向量化展开了循环而变长，RV64 因为化简而变短。本实验只观察 IR 与汇编，没有测量运行耗时、代码体积或能耗，因此不据此声称任何量化的性能提升。

8 MLIR / AscendNPU VecAdd

MLIR 以 operation/value/block/region 和 dialect 描述不同抽象层。多层表示的作用是让每一阶段保留它需要的信息：张量形状和算子边界便于做 fusion、tiling 与调度；进一步 lowering 后，内存位置、数据搬运和目标硬件操作才逐步明确。若过早把算子摊平成低层指令，后续优化就难直接利用张量和算子语义。LLVM IR 可作为后续通用低层表示，但它不替代前面针对张量和 NPU 硬件的抽象。[1, 2, 3, 4]

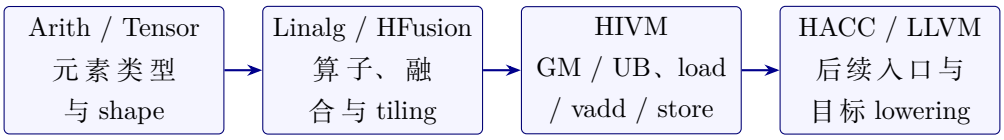


图 4: VecAdd 的概念 lowering 主线；本次逐行核对的官方文件位于 HIVM 层

表 11: VecAdd lowering 各层表达的信息

层	VecAdd 表达	关注点
Arith/Tensor	张量形状、元素类型和逐元素加	算法语义，未指定 NPU 地址空间
Linalg/HFusion	张量算子及融合、tiling	算子组合和执行计划
HIVM	GM/UB、load、vadd、store	显式表示数据搬运和 NPU 运算位置
HACC	entry 与 HOST/DEVICE 属性	主机/设备边界与 kernel 入口
LLVM IR/后端	更低层通用表示和目标代码	后续 lowering；此处不是本次实测 dump

官方 HIVM VecAdd 文件 `mlir/sources/vecadd_hivm.mlir` 中，16 个 `i16` 元素的 A、B、C 参数位于 GM；两次 `hivm.hir.load` 把输入搬到 UB，`hivm.hir.vadd` 在 UB 上计算，`hivm.hir.store` 再写回 GM。这里的数据不是只标注为“向量值”，还带有存储位置和搬运路径。

```
func.func @add(%arg0: memref<16xi16, #hivm.address_space<gm>>,
               %arg1: memref<16xi16, #hivm.address_space<gm>>,
               %arg2: memref<16xi16, #hivm.address_space<gm>>)
    attributes {hacc.entry, hacc.function_kind = #hacc.function_kind<DEVICE>} {
  %alloc   = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.load ins(%arg0 : ...<gm>) outs(%alloc   : ...<ub>)   // GM -> UB
  %alloc_0 = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.load ins(%arg1 : ...<gm>) outs(%alloc_0 : ...<ub>)   // GM -> UB
  %alloc_1 = memref.alloc() : memref<16xi16, #hivm.address_space<ub>>
  hivm.hir.vadd ins(%alloc, %alloc_0 : ...<ub>, ...<ub>) outs(%alloc_1 : ...<ub>)
  hivm.hir.store ins(%alloc_1 : ...<ub>) outs(%arg2 : ...<gm>)  // UB -> GM
  return
}
```

与高层表示相比，这里的 `memref.alloc` 都显式标注 `<ub>` 地址空间，`hivm.hir.load/store` 把“从哪搬到哪”写成显式指令。这正是 NPU 编程与普通标量程序的关键差别：计算必须在片上存储（UB）进行，数据必须先从全局内存（GM）搬入、算完再搬回，因此原本隐含的硬件执行策略被编码进了 IR。[4, 6]

8.1 PlanMemory 上游用例分析

PlanMemory 根据 buffer 生命周期和硬件约束规划片上地址。它把局部 `memref.alloc` 改写为带偏移的 `hivm.hir.pointer_cast`。

Ascend 上游测试文件 `plan-memory.mlir` 中的 `test_mem_allocate_basic` 用例包含四个大小为 $16 \times 16 \times 16 \times 2 = 8192$ 字节的 UB 临时 buffer。其上游 RUN 命令与关键断言为：

```
// RUN: bishengir-opt %s -hivm-plan-memory -split-input-file -verify-diagnostics |
    FileCheck %s
// CHECK-NOT: memref.alloc()
```

```
// CHECK: hivm.hir.pointer_cast ... offset = 0
```

CHECK-NOT 断言输出不再包含 memref.alloc()，CHECK 要求这些 buffer 被改写为带偏移的 pointer_cast。这说明 Pass 的作用不是改变计算语义，而是把“我需要一块 UB”这一抽象请求落实为“这块 UB 在各硬件缓冲区中的具体地址”。[8, 9, 10]

8.2 观察范围与证据边界

第 8 节与前七节的性质不同，必须明确区分“本机真实运行得到”和“依据官方材料分析得到”两类内容。

本机环境检查的实际结果是：系统中没有 CMake，Ninja 版本为 1.10.1，低于 AscendNPU-IR 官方构建要求的 1.12。按任务书中“环境不满足即退化为资料与源码分析”的约定，本实验没有克隆或构建 bishengir-opt，也没有运行任何 HIVM Pass。因此：

- 本节的真实文件是 mlir/sources/vecadd_hivm.mlir（官方 16×i16 VecAdd）与 mlir/sources/selected_tests/plan_memory_input.mlir（官方 plan-memory.mlir 的相关片段），二者来自上游仓库。
- 其中的 RUN/CHECK 是上游测试的期望，不是本机产生的输出；mlir/outputs/ 目录下没有 Before/After 文件，因为 Pass 未被执行。
- 层次图（Arith/Tensor → HFusion → HIVM → HACC/LLVM）是根据官方架构资料整理的概念主线；可以逐行核对的实物只到 HIVM 层。
- 没有安装完整 CANN，没有配置 Ascend 驱动，没有生成或运行 kernel .o，没有在真实 NPU 上执行。

之所以仍然保留这一节，是因为它要回答的问题——“为什么需要多层 IR”“硬件细节何时进入 IR”——在读完 HIVM 层的真实 IR 与 PlanMemory 的官方断言后已经可以回答清楚。但本报告不把这段分析表述为一次完整的编译实验结果，也不声称获得了任何 NPU 运行数据。[11, 7]

9 实验可复现性

全部实验由 scripts/ 下的脚本驱动，每个阶段一条命令即可重放，输出同时写入 logs/ 与 records/。自动化与手写产物分属不同目录：generated/ 只放工具生成结果，llvm/ 与 riscv/ 放人工编写结果，避免把编译器输出当成交付内容。

表 12: 各阶段复现入口

脚本	作用
00_environment_check.sh	记录系统与工具版本，作为环境快照
01_reference.sh	编译宿主机 Ground Truth 并跑六组测试
02_pipeline.sh	生成 .i / .ll / .s / .o 并显式链接
03_llvm_analysis.sh、03_llvm_validation.sh	抽取 IR 做 CFG 分析；验证手写 manual.ll
04_riscv_reference.sh、05_riscv_validation.sh	生成 RV64 参考汇编；汇编链接手写 manual.s 并在 QEMU 运行
06_optimization.sh	生成 O0/O2 三种目标的对照与 diff
08_mlir_exploration.sh	记录进阶部分的只读环境检查与范围判定
09_final_validation.sh	三路实现统一比对并输出总表
10_build_report.sh	用 Tectonic 编译本报告

统一执行形式为 bash scripts/<name>.sh 2>&1 | tee logs/<name>.log。脚本以 set -euo pipefail 开头，任一步失败即中止，因此 logs/ 中的记录不会掩盖中途错误。实验前需先 source ../env.sh 以取得 QEMU 与 RISC-V 工具链路径；09_final_validation.sh 已内置该步骤。

10 结论

本文用一个包含函数、循环、分支、取模和运行库调用的 SysY 程序，走通了从源程序到可执行文件的全部环节，并在此基础上独立完成了两套手写实现。按引言提出的四个问题，可以得到以下结论。

关于四个阶段的分工。预处理只做文本替换，产物仍是源文本；前端把它变成 LLVM IR，其间经过词法、语法和语义分析；汇编器把汇编变成可重定位目标文件，此时地址未定、外部符号未解；链接器才把定义与引用配对并定下最终地址。本实验的 example.o 是 ET_REL、入口地址为 0、含两个未定义符号，而链接后的文件六组测试全部通过，这一对比即为四者职责的直接证据。

关于 LLVM IR 到源代码的映射。-O0 下 IR 与源程序接近一一对应：alloca/load/store 表示变量，icmp/br 表示比较与转移，define/call/ret 表示函数关系。while 和 if 都没有专用指令，它们由基本块、条件计算和 br 组合而成；循环由 if.end 跳回 while.cond 的回边体现。正因为这一映射是结构性的，才能在阅读自动生成结果之后独立重写出一份语义等价、命名

更清晰的 `manual.ll`。

关于 RISC-V 的落地方式。参数与返回值共用 `a0`，`calc` 用 32 字节栈帧保存 `ra`、`s0` 和三个局部量，控制流落为比较加条件跳转的标签结构。值得记录的是，手写版与 Clang 自动版收敛到了*相同的栈帧布局*，但在取模和 `main` 的栈帧大小上做了不同取舍——说明 ABI 约束是刚性的，而具体实现仍留有空间。

关于优化与抽象层次。`-O2` 先改变值的表示（栈槽变 SSA，出现 32 个 `phi`），再重写算术（奇偶判断退化为变长左移，整个 `if/else` 消失），最后才由后端按目标能力决定是否向量化：x86-64 生成四 lane 运算和 `paddd`，RV64 因缺少 RVV 仍是标量循环。两种目标的汇编行数一增一减（87→385 与 91→56），说明行数不能作为优化效果的度量。MLIR 部分则从另一个方向回答了分层问题：HIVM 层把 GM/UB 地址空间和 `hivm.hir.load/vadd/store` 显式写入 IR，把原本隐含的硬件搬运策略变成可分析、可规划的信息，这正是“多层 IR”相对单一低层 IR 的价值所在。

本实验的边界。所有前七节结论均基于本机真实产物，可用第 9 节的脚本逐阶段复现。第 8 节的 MLIR/Ascend 部分受限于环境（无 CMake、Ninja 1.10.1 低于要求的 1.12），为官方文档与上游测试用例的结构分析，没有运行 `bishengir-opt`，没有生成或运行 kernel，也没有在 Ascend NPU 上执行；报告中不对此作任何实测声明。

参考文献

[1] LLVM Project. *MLIR Language Reference*. https://mlir.llvm.org/docs/LangRef/（访问日期：2026-10-07）.

[2] LLVM Project. *MLIR Rationale*. https://mlir.llvm.org/docs/Rationale/Rationale/（访问日期：2026-10-07）.

[3] LLVM Project. *Targeting LLVM IR from MLIR*. https://mlir.llvm.org/docs/TargetLLVMIR/（访问日期：2026-10-07）.

[4] Ascend. *AscendNPU-IR Framework Integration*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/developer_guide/conversion/framework_interface.md（访问日期：2026-10-07）.

[5] Ascend. *AscendNPU-IR Interface API*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/developer_guide/conversion/interface_api.md（访问日期：2026-10-07）.

[6] Ascend. *HIVM VecAdd add.mlir*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/add.mlir（访问日期：2026-10-07）.

[7] Ascend. *HIVM VecAdd README and host wrapper*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/README.md（访问日期：2026-10-07）.

[8] Ascend. *Plan Memory*. https://ascendnpu-ir.gitcode.com/en/developer_guide/features/PlanMemory/PlanMemory.html（访问日期：2026-10-07）.

[9] Ascend. *HIVM Passes*. https://ascendnpu-ir.gitcode.com/en/developer_guide/passes/hivm_passes.html（访问日期：2026-10-07）.

[10] Ascend. *PlanMemory Test Case*. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Dialect/HIVM/plan-memory.mlir（访问日期：2026-10-07）.

[11] Ascend. *AscendNPU-IR Build and Installation*. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/introduction/quick_start/installing_guide.md（访问日期：2026-10-07）.

[12] Compiler-Labs. 编译原理作业要求与执行流程（本地课程文档，访问日期：2026-10-07）.